

23 . DECISION TREE LEARNING USING ID3

Aim

To implement the ID3 decision tree learning algorithm from scratch using entropy and information gain and construct a decision tree from a given dataset.

Algorithm + Procedure

1. Read the training dataset from the CSV file.
2. Calculate the entropy of the target attribute.
3. Calculate the information gain for every available attribute.
4. Select the attribute having the highest information gain.
5. Make the selected attribute the root node of the decision tree.
6. Divide the dataset according to the values of the selected attribute.
7. Recursively apply the same process to each subset.
8. If all examples in a subset belong to the same class, create a leaf node.
9. If no attributes remain, assign the majority class as the leaf.
10. Display the final decision tree.

Code

```
import pandas as pd
import numpy as np
# Create PlayTennis dataset
data = {
    'Outlook': [
        'Sunny', 'Sunny', 'Overcast', 'Rain',
        'Rain', 'Rain', 'Overcast', 'Sunny',
        'Sunny', 'Rain', 'Sunny', 'Overcast',
        'Overcast', 'Rain'
    ],
    'Temperature': [
        'Hot', 'Hot', 'Hot', 'Mild',
        'Cool', 'Cool', 'Cool', 'Mild',
```

```

        'Cool', 'Mild', 'Mild', 'Mild',
        'Hot', 'Mild'
    ],
    'Humidity': [
        'High', 'High', 'High', 'High',
        'Normal', 'Normal', 'Normal', 'High',
        'Normal', 'Normal', 'Normal', 'High',
        'Normal', 'High'
    ],
    'Wind': [
        'Weak', 'Strong', 'Weak', 'Weak',
        'Weak', 'Strong', 'Strong', 'Weak',
        'Weak', 'Weak', 'Strong', 'Strong',
        'Weak', 'Strong'
    ],
    'PlayTennis': [
        'No', 'No', 'Yes', 'Yes',
        'Yes', 'No', 'Yes', 'No',
        'Yes', 'Yes', 'Yes', 'Yes',
        'Yes', 'No'
    ]
}

# Create CSV file
df = pd.DataFrame(data)
df.to_csv('playtennis.csv', index=False)

# Calculate Entropy
def entropy(col):
    values, counts = np.unique(col, return_counts=True)
    probabilities = counts / counts.sum()

```

```

    return -np.sum(
        probabilities * np.log2(probabilities)
    )
# Calculate Information Gain
def info_gain(data, attr, target):
    total_entropy = entropy(data[target])
    values, counts = np.unique(
        data[attr],
        return_counts=True
    )
    weighted_entropy = 0
    for i in range(len(values)):
        subset = data[data[attr] == values[i]]
        weighted_entropy += (
            counts[i] / counts.sum()
        ) * entropy(subset[target])
    return total_entropy - weighted_entropy
# ID3 Algorithm
def id3(data, attributes, target):
    # If all examples belong to one class
    if len(np.unique(data[target])) == 1:
        return data[target].iloc[0]
    # If no attributes remain
    if not attributes:
        return data[target].mode()[0]
    # Calculate information gain
    gains = {}
    for attr in attributes:
        gains[attr] = info_gain(

```

```

        data, attr, target
    )
    # Select attribute with highest gain
    best = max(gains, key=gains.get)
    tree = {best: {}}
    # Create branches
    for value in np.unique(data[best]):
        subset = data[data[best] == value]
        remaining = [
            attr for attr in attributes
            if attr != best
        ]
        tree[best][value] = id3(
            subset,
            remaining,
            target
        )
    return tree

# Read CSV file
df = pd.read_csv('playtennis.csv')
attributes = list(df.columns[:-1])
tree = id3(
    df,
    attributes,
    df.columns[-1]
)
print("Decision Tree:")
print(tree)

```

Output

Decision Tree:
{ 'Outlook': { 'Overcast': 'Yes', 'Rain': { 'Wind': { 'Strong': 'No', 'Weak': 'Yes' } }, 'Sunny': { 'Humidity': { 'High': 'No', 'Normal': 'Yes' } } } }

Result:

The ID3 decision tree algorithm was successfully implemented using entropy and information gain, and the decision tree was constructed correctly using the PlayTennis dataset.